基于 BERT 的时序感知模型对抗攻击与鲁棒性分析开题报告

方向 10 小组 B

杨蕊竹、姚欣煜、徐梦茹、陆鹏宇、井立帅

一、 研究背景与意义

1. 研究背景

随着深度学习技术的演进，自然语言处理 NLP 模型已成为众多关键软件系统的核心组件。然而，这些系统在面对精心构造的对抗样本时表现出了严重的安全隐患。本项目聚焦于软件质量保障领域中的模型鲁棒性验证问题，探讨序列感知模型在面对输入扰动时的预测稳定性。

鉴于当前 NLP 领域已全面迈入预训练大模型时代，传统的 LSTM 由于在长距离依赖捕获上的局限性，已逐渐被基于 Transformer 的模型取代。因此，本项目将目标模型升级为当前工业界广泛采用的 BERT。文本数据本质上属于一种高度结构化的时序序列数据。正如 Jin 等人在提出典型黑盒攻击算法 TextFooler 时所指出的："尽管预训练语言模型在多项自然语言处理任务中取得了巨大成功，但它们对对抗性样本仍然表现出极大的脆弱性，仅需替换文本序列中的极少数字词，即可导致模型产生严重误判。"[1] 这种脆弱性直接威胁到基于大语言模型开发的软件系统的可靠性。因此，探究 BERT 模型在面对特定时序扰动，比如关键序列步替换下的表现，不仅符合时序感知模型安全验证的理论要求，更是当前软件质量保障的迫切需求。

2.研究意义

本项目的理论意义主要体现在对序列微扰与关键时间步在现代自注意力机制下的重新定义与攻击策略改进：

揭示自注意力机制下的关键时间步脆弱模式。在传统的循环神经网络中，攻击者通常依靠门控状态来寻找时间维度的脆弱点。而在 BERT 的架构中，Token 序列的依赖关系由自注意力矩阵决定。Ebrahimi 等人在提出白盒攻击算法 HotFlip 时强调："通过计算输入向量的损失梯度并利用方向导数，我们可以精确评估单个特征词被替换后对模型整体损失函数的影响程度。"[2] 本项目将

以此为理论基础，结合 BERT 的梯度信息与注意力热力图，精准定位序列中的关键时间步，探究其在遭受微扰时的语义崩溃临界点。

探索并改进文本序列的对抗攻击策略。本项目不局限于对现有算法的简单复现。针对现有基线攻击算法通常采用固定约束阈值，如绝对的余弦相似度下限，容易导致攻击初期效率低下或强行攻击导致语义失真的局限性，本项目计划在 TextFooler 等基线算法上进行改进，提出一种约束自适应动态调整策略。该策略根据攻击迭代次数或状态变化，自适应调节语义相似度等约束强度：在攻击探索阶段采用宽松约束以提升搜索效率，在收敛阶段动态收紧约束以保证对抗样本的质量。这一轻量级改进为自然语言时序序列的对抗生成理论提供了兼顾效率与保真度的新优化思路与实证支持。

二、 相关现状调研

自然语言处理领域中的对抗攻击研究起步稍晚于计算机视觉，其核心难点在于文本数据的离散性，使得传统的基于连续梯度的微小扰动难以直接应用。随着预训练大语言模型，如 BERT、RoBERTa 等的广泛部署，探究这些时序感知模型在对抗扰动下的鲁棒性，已成为学术界与工业界的共识。综合近年的前沿文献，目前的现状调研可归纳为以下三个关键维度：

1. 主流对抗攻击范式的演进

当前针对文本序列的对抗攻击主要分为白盒攻击 White-box 与黑盒攻击 Black-box 两大范式：

白盒攻击：攻击者拥有目标模型的全部内部信息，如参数、梯度。Ebrahimi 等人提出的 HotFlip 算法是该领域的典型代表，其通过计算损失函数对输入序列的一阶方向导数，精准定位并替换“关键时间步”。然而，此类方法往往会破坏句法结构，生成缺乏可读性的乱码。

黑盒攻击：攻击者仅能获取模型的输入输出接口。Jin 等人提出的 TextFooler 算法早期通过同义词词典进行暴力替换，奠定了黑盒攻击的基线。为了进一步提升对抗样本的上下文自然度，Li 等人提出了一种针对 BERT 的对抗攻击范式 BERT-Attack。该方法创新性地利用了 BERT 自身的掩码语言模型、机制来生成上下文相关的替换词。相比于基于静态词向量的传统方法，其生成的对抗样本更

符合语境，在保证高攻击成功率的同时显著提升了语义一致性 [6]。本项目在后续的评估环节，亦将此类先进的基线算法纳入对比验证。

2. 对抗生成策略的搜索空间与框架化

对抗攻击的本质是在庞大的词汇组合空间中寻找最优扰动。在被广泛引用的基准测试研究《Searching for a Search Method》中指出，文本对抗攻击本质上是一个高度复杂的组合优化问题[4]。该论文系统性地对比了贪心搜索、集束搜索、以及遗传算法等多种策略，发现不同的搜索算法在攻击成功率与计算效率之间存在显著的权衡、，并不存在绝对的通用最优解。这一结论为 Morris 等人后续开发的模块化开源测试框架 TextAttack 的“目标-搜索-变换-约束”四层架构设计奠定了坚实的理论基础 [3]。本项目在初步搭建实验环境时，正是依托该模块化思想，实现了基线算法的快速实例化。

3. 样本质量评估体系的重构与现有局限性

在追求高攻击成功率 ASR 的早期阶段，学界逐渐暴露出对抗样本语义失真的严重问题。Morris 等人在另一项关于对抗样本重评估的研究《Reevaluating Adversarial Examples in Natural Language》中提出了严厉的质疑 [5]。他们指出，许多宣称攻击成功的算法，其生成的文本在语义一致性与人类可读性上存在严重瑕疵。该研究呼吁，高质量的软件鲁棒性评估必须引入更严格的约束指标。如高阈值的语义相似度、语言流畅性困惑度等，推动了对抗攻击向高质量保真样本维度的演进。

4. 现有研究的局限性与本项目切入点

基于上述调研，我们发现现有主流开源框架在处理搜索效率与高质量保真的矛盾时，通常采用静态硬编码约束（例如规定全程的余弦相似度必须始终 >0.8）。这种一刀切的策略具有明显的局限性：在搜索初期限制了解空间的探索，导致大量无效计算；而在搜索末期又难以进一步逼近真实语义的边界。

为此，本项目旨在打破这一瓶颈，从约束机制动态化切入，提出约束自适应动态调整策略，以此作为提升时序感知预训练模型鲁棒性测试效率的新优化路径。

三、 初步搭建情况

1. 实验环境与工具链配置

为了确保实验的可复现性与环境隔离，本项目采用了 Miniconda 作为环境管理工具，构建了专属的虚拟开发环境：

操作系统: Windows 11 (x64)

环境管理: Miniconda (conda)

虚拟环境名称: bert_attack

Python 版本: 3.9.25 (选定该版本以确保与主流深度学习库的最佳兼容性)

IDE/编辑器: Visual Studio Code (已配置 Python 插件及调试器 launch.json)

```
(base) C:\Users\meng>conda create -n bert_attack python=3.9 -y
3 channel Terms of Service accepted
Channels:
 - defaults
Platform: win-64
Collecting package metadata (repodata.json): done
Solving environment: done


==> WARNING: A newer version of conda exists. <==
    current version: 26.1.1
    latest version: 26.3.1

Please update conda by running

    $ conda update -n base -c defaults conda



## Package Plan ##

  environment location: D:\minniconda\envs\bert_attack

  added / updated specs:
    - python=3.9


The following packages will be downloaded:

    package                    |            build
    ---------------------------|-----------------
    bzip2-1.0.8                |       h2bbff1b_6          90 KB
    ca-certificates-2026.3.19  |       haa95532_0         126 KB
    expat-2.7.5                |       hd7fb8db_0          32 KB
    libexpat-2.7.5             |       hd7fb8db_0         120 KB
    libffi-3.4.4               |       hd77b12b_1         122 KB
    libzlib-1.3.1              |       h02ab6af_0          64 KB
    openssl-3.5.5              |       hbb43b14_0         8.9 MB
    pip-26.0.1                 |     pyhc872135_1         1.1 MB
    python-3.9.25              |       h716150d_1        16.0 MB
    setuptools-80.9.0          |   py39haa95532_0         1.4 MB
    sqlite-3.51.2              |       hee5a0db_0         917 KB
    tk-8.6.15                  |       hf199647_0         3.5 MB
    tzdata-2026a               |       he532380_0         117 KB
    ucrt-10.0.22621.0          |       haa95532_0         620 KB
    vc-14.3                    |      h2df5915_10          19 KB
    vc14_runtime-14.44.35208   |      h4927774_10         825 KB
    vs2015_runtime-14.44.35208 |      ha6b5a95_10          19 KB
    wheel-0.45.1               |   py39haa95532_0         145 KB
    xz-5.8.2                   |       h53af0af_0         265 KB
    zlib-1.3.1                 |       h02ab6af_0         113 KB
    ------------------------------------------------------------
                                           Total:        34.5 MB

The following NEW packages will be INSTALLED:

  bzip2              pkgs/main/win-64::bzip2-1.0.8-h2bbff1b_6
  ca-certificates    pkgs/main/win-64::ca-certificates-2026.3.19-haa95532_0
  expat              pkgs/main/win-64::expat-2.7.5-hd7fb8db_0
  libexpat           pkgs/main/win-64::libexpat-2.7.5-hd7fb8db_0
  libffi             pkgs/main/win-64::libffi-3.4.4-hd77b12b_1
  libzlib            pkgs/main/win-64::libzlib-1.3.1-h02ab6af_0
  openssl            pkgs/main/win-64::openssl-3.5.5-hbb43b14_0
  pip                pkgs/main/noarch::pip-26.0.1-pyhc872135_1
  python             pkgs/main/win-64::python-3.9.25-h716150d_1
  setuptools         pkgs/main/win-64::setuptools-80.9.0-py39haa95532_0
  sqlite             pkgs/main/win-64::sqlite-3.51.2-hee5a0db_0
  tk                 pkgs/main/win-64::tk-8.6.15-hf199647_0
  tzdata             pkgs/main/noarch::tzdata-2026a-he532380_0
  ucrt               pkgs/main/win-64::ucrt-10.0.22621.0-haa95532_0
done
#
# To activate this environment, use
#
#     $ conda activate bert_attack
#
# To deactivate an active environment, use
#
#     $ conda deactivate
```

2. 核心技术栈选型与安装

放弃了早期的 TensorFlow 方案，全面转向性能更优、社区支持更广的 PyTorch + Hugging Face 生态系统。目前已成功安装并验证以下核心库：

深度学习框架: PyTorch 2.11.0 ，目前配置为 CPU 模式，后续可根据硬件条件无缝切换至 CUDA 加速

模型库: Transformers (Hugging Face)，用于加载并调用预训练的 BERT 系列模型，如 bert-base-uncased。

数据集库: Datasets，用于高效加载 IMDB、MRPC 等标准 NLP 评测数据集

对抗攻击框架: TextAttack，该库集成了本项目计划复现的 TextFooler 与 HotFlip 等核心攻击算法原型

```
(base) C:\Users\meng>conda create -n bert_attack python=3.9 -y
3 channel Terms of Service accepted
```

```
(base) C:\Users\meng>conda activate bert_attack

(bert_attack) C:\Users\meng>pip install torch torchvision torchaudio transformers datasets textattack
Collecting torch
  Downloading torch-2.8.0-cp39-cp39-win_amd64.whl.metadata (30 kB)
Collecting torchvision
```

3. 核心组件加载与可用性验证

已编写并运行环境自动化校验脚本（main.py），验证结果如下：

模型加载验证: 成功从 Hugging Face Model Hub 自动下载并实例化 bert-base-uncased 预训练权重及对应的 Tokenizer。验证了模型张量输出维度符合预期。

对抗攻击库验证: 成功初始化 Textattack 及其相关 NLP 依赖，包括 NLTK 数据包、词性标注器等，确保了后续攻击脚本的即插即用。

调试链路验证: 在 VS Code 中完成了基于 {workspaceFolder} 路径变量的运行配置，支持通过命令行参数动态调整实验参数（如任务名称、批处理大小等）。

```
D:\minniconda\envs\bert_attack\lib\site-packages\jieba\_compat.py:18: UserWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/e
n/latest/pkg_resources.html. The pkg_resources package is slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<
81.
  import pkg_resources
 环境检查开始...
 PyTorch 版本: 2.8.0+cpu
   GPU 状态:  不可用, 将使用 CPU

正在从Hugging Face加载核心组件: 'bert-base-uncased'...
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized: ['classifier.
bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
   Hugging Face Transformers:  加载成功!

检查对抗攻击库 TextAttack...
   TextAttack: 导入完成!
所有环境依赖已全部就位
(venv) PS D:\aiCode\MyBERTProject>
```

main.py:

```
import torch
from transformers import BertTokenizer, BertForSequenceClassification
import textattack
def final_check():
    print(" 环境检查开始...")
    # 1. 检查 PyTorch 和 GPU
    print(f" PyTorch 版本: {torch.__version__}")
    if torch.cuda.is_available():
        print("    GPU 状态:    可用")
    else:
        print("    GPU 状态:    不可用, 将使用 CPU ")
    # 2. 检查 Transformers (Hugging Face)
    try:
        model_name = "bert-base-uncased"
        print(f"\n 正在从 Hugging Face 加载核心组件: '{model_name}'...")
        tokenizer = BertTokenizer.from_pretrained(model_name)
        model = BertForSequenceClassification.from_pretrained(model_name)
        print("    Hugging Face Transformers:   加载成功！ ")
    except Exception as e:
        print(f"    Hugging Face Transformers 加载失败: {e}")
        return
    # 3. 检查 TextAttack (你们项目后续的攻击库)
    print(f"\n 检查对抗攻击库 TextAttack...")
    # 既然程序能运行到这里，就说明 textattack 已经导入成功了
    print("    TextAttack: 导入完成！ ")
    print("所有环境依赖已全部就位")
if __name__ == "__main__":
    final_check()
```

4. TextAttack 的核心功能解析

为落实针对现有工具有改进的最终研究目标，本项目在成功搭建环境后，对所选用的核心开源安全评估框架 TextAttack 的底层源码与架构进行了深度的功能解析。经解析梳理，TextAttack 框架摒弃了传统攻击算法的硬编码模式，创新性地采用了一种高度模块化的四层解耦架构。这种面向对象的设计模式将任意复杂的文本对抗攻击拆解为四个独立且可替换的组件，这也是我们在后续研究中能够实施自适应动态约束改进的基础：

目标函数（Goal Functions）：该模块负责界定本次攻击的成功条件与优化方向。在本项目的情感分类任务中，主要解析并调用了 UntargetedClassification 无目标分类误导功能，即只要能使 BERT 模型的输出偏离原始正确标签即视为攻击成功。

转换规则（Transformations）：该模块定义了在“关键时间步”上具体实施的微扰动作。本项目重点解析了 WordSwap 词元替换相关类，它规定了算法是基于词向量，如 GloV 还是基于掩码语言模型来生成候选同义词集。

约束条件（Constraints）：（本项目重点改进对象） 该模块充当了对抗样本质量的守门员，决定了生成的扰动文本是否符合人类的自然语言习惯。其核心功能包括基于 Universal Sentence Encoder 的语义相似度约束、基于 NLTK 的词性（POS）一致性约束等。在解析过程中我们发现，该工具默认采用静态硬编码阈值（如相似度固定 > 0.8），这为本项目后续引入动态约束调节机制留下了清晰的切入点。

搜索算法（Search Methods）：该模块负责在庞大的文本扰动空间中寻找最优解。我们解析了工具内置的贪心搜索 Greedy Search 与词汇重要性排序 WIR 等策略，它们通过计算模型前向传播的概率下降梯度，为每一步的词元替换提供方向指引。

通过对上述核心功能的成功解析，本项目已彻底理清了深度学习模型文本对抗攻击的工程流转逻辑。这种模块化的工具架构使我们能够在不破坏原有搜索框架与目标函数的前提下，独立重构 Constraint 约束类，从而实现代码层面的算法升级。

5. 阶段性进展总结

目前，实验平台已具备“预训练模型加载 -> 数据集预处理 -> 对抗攻击库调用”的完整闭环。环境搭建阶段已圆满结束，项目已完全具备进入下一阶段：基线攻击算法复现的技术条件。

四、 进度安排与预期成效

1.进度安排

时间段	主要任务	预期产出
4.5 之前	文献综述与环境搭建	完成相关文献整理，搭建 BERT 开发环境（Python+PyTorch+Transformers）
4.6-4.12	复现基线攻击算法（TextFooler、HotFlip）	获得可运行的攻击基线代码，验证在 BERT 上初步效果
4.13-4.19	设计并实现约束自适应动态调整策略	完成改进版攻击算法原型，支持语义相似度阈值的动态调节
4.20-4.26	在标准数据集（如 IMDB）上进行实验	获取攻击成功、对抗样本语义保真度等关键指标
4.27-5.3	鲁棒性分析与对比实验	对比改进算法与基线在 BERT 上的鲁邦性提升效果
5.4-5.10	结果可视化与论文撰写	生成注意力热力图，攻击效率曲线等图表，完成实验部分写作
5.11-	整体修改与答	提交完整开题报告、项目文档及答辩 PPT

5.17	辩准备	

2. 预期成效

本项目预期在理论与实证两个层面取得如下成效：

揭示 BERT 在时序扰动下的脆弱模式 通过梯度信息与注意力热力图，定位 BERT 中易受攻击的关键时间步，总结其语义崩溃临界点规律。

提出一种改进的对抗攻击算法 在 TextFooler 等基线基础上，设计并实现约束自适应动态调整策略，在攻击效率与对抗样本语义保真度之间取得更优平衡。

量化评估 BERT 的鲁棒性 在多个标准文本分类数据集上，对比改进算法与基线方法的攻击成功率 ASR、对抗样本的余弦相似度与语法困惑度 PPL 等指标，验证所提策略的有效性。

形成可复现的代码与实验平台 提供完整的攻击与鲁棒性评估代码，为后续时序感知模型的安全性研究提供可复现的基准与工具支持。

完成高质量的项目报告与答辩展示 形成结构完整、实验详实的课程项目报告，并准备相应的答辩材料，清晰展示研究思路、实现过程与主要结论。

五、 参考文献

[1] Jin D, Jin Z, Zhou JT, Szolovits P. Is BERT really robust? A strong baseline for natural language attack on text classification and entailment. In: Proc. of the 34th AAAI Conf. on Artificial Intelligence. Palo Alto: AAAI Press, 2020. 8018–8025.

[2] Ebrahimi J, Rao A, Lowd D, Dou D. HotFlip: White-box adversarial examples for text classification. In: Proc. of the 56th Annual Meeting of the Association for Computational Linguistics. Melbourne: Association for Computational Linguistics, 2018. 31–36.

[3] Morris JX, Lifland E, Yoo JY, Grigsby J, Jin D, Qi Y. TextAttack: A framework for adversarial attacks, data augmentation, and adversarial training in NLP. In: Proc. of the 2020 Conf. on Empirical Methods in Natural Language Processing: System Demonstrations. Online: Association for Computational Linguistics, 2020. 119–126.

[4] Morris JX, Lifland E, Yoo JY, Qi Y. Searching for a search method: Benchmarking search algorithms for generating NLP adversarial examples. In: Proc. of the 2020 Conf. on Empirical Methods in Natural Language Processing (EMNLP). Online: Association for Computational Linguistics, 2020. 1766–1779.

[5] Morris JX, Lifland E, Yoo JY, Qi Y. Reevaluating adversarial examples in natural language. In: Findings of the Association for Computational Linguistics: EMNLP 2020. Online: Association for Computational Linguistics, 2020. 3829–3839.

[6] Li L, Ma R, Guo Q, Xue X, Qiu X. BERT-Attack: Adversarial attack against BERT using BERT. In: Proc. of the 2020 Conf. on Empirical Methods in Natural Language Processing (EMNLP). Online: Association for Computational Linguistics, 2020. 6193–6202.